

Decoder: A Two-step Approach

Tao Liu, Yilan Luo, Chuqi Wang and Hualong Zhang
(Team Decoder)

Dept. of Bioengineering, Imperial College London, London, UK
email: tl622@ic.ac.uk, yl6822@ic.ac.uk, cw822@ic.ac.uk, hz1922@ic.ac.uk

Abstract—This report uses a two-step approach to predict hand movement positions in monkeys based on neural activity. A Naive Bayes classifier is used to classify orientations, followed by an orientation-specific Feedforward Neural Networks regressor for real-time hand position prediction. The method completes the task in 23s with a root mean square error of 26.02985.

Keywords—two-step; monkey; Naive Bayes; Feedforward Neural Networks;

I. INTRODUCTION

The movement trajectory decoding problem has been well attempted with many angles of approaches. The improvement of the neural recording techniques, single-unit and multiunit isolation for spike sorting, the preprocessing of neural spike train, time series analyzing, feature extraction[1] and the parameter and model selection[2] of decoding would all impact on the prediction accuracy. The theoretical connection of the machine decoding to the brain encoding process was revealed and supported by the success of predicting and driving the motion from the neural recordings. In order to understand the potential of exploring the hidden layers in the mammal brain and the limitation of well performed linear method, it is worth to study the experimental data and evaluate the variation in performance when combine different models and algorithms to solve the decoding task.

In this report we are required to predict the hand position of monkey based on the recorded spike trains by using MATLAB. The dataset demonstrated that the monkey executed arm movements at equidistant angles along eight directions, across multiple repeated trials.

Previous studies suggested that the directional classification features extracted from the neural activity are considered as independent, relatively transparent and capable of linear separation[3]. Machine learning algorithms are shown as a powerful tool of decoding[2], as the input dataset, time continuous spike trains containing features in a large data size and the multiple trials experiment requires a large memory space for storage. Therefore, the solution proposed in this report applies a Naive Bayes (NB) direction classifier to the pre-movement period neural activity, and the output directional label is then used to choose the Feedforward Neural Networks (FNN) regressor that trained for each direction. The results manifest that our decoder allows acceptable prediction of the monkey hand trajectories based on real-time neural recordings with small amount of computational resources costing.

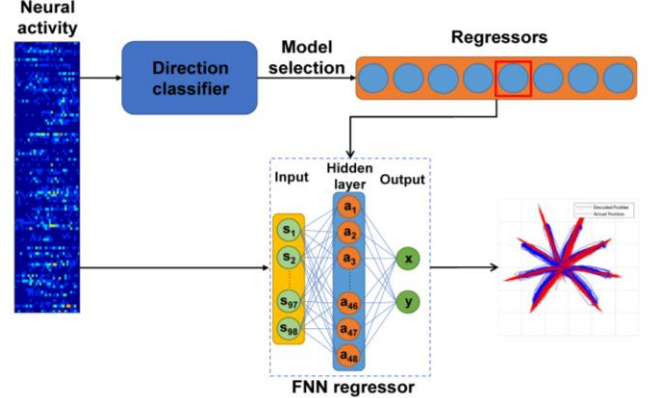


Figure 1. Framework of employed method

II. METHODS

The report proposes a two-part solution: firstly, a pre-motor neural activity based 8-class classification is performed to recognize the direction of hand movement, and secondly, 8 regressors are trained to predict real-time hand position from neural activity. The input is first classified for direction and the corresponding regressor is selected. The input is then sent to the corresponding regressor to predict hand position. The framework is shown in Fig. 1.

A. Direction Recognition

The Naive Bayes classifier is an easy and efficient classifier that relies on the Bayes theorem and has the advantage of fast learning and testing[4]. It is mainly used to classify items by comparing the probabilities of the items on different class.

In general, we assume that each feature is conditionally independent and is not influenced by potential factors in the final prediction process[4]. $X = \{a_1, a_2, \dots, a_m\}$, and classes $\{Y_1, Y_2, \dots, Y_n\}$. We derive (1) According to the Bayes theorem[4].

$$P(Y_i|X) = P(X|Y_i)P(Y_i)/P(X) \quad (1)$$

The $P(X)$ of the denominator in (1) has the same meaning to all classification. This means $P(X)$ can be ignored during the comparison. Thus only the numerator part of (2) is needed to be compared.

$$P(X|Y_i)P(Y_i) = P(Y_i) \prod_{j=1}^m P(a_j|Y_i) \quad (2)$$

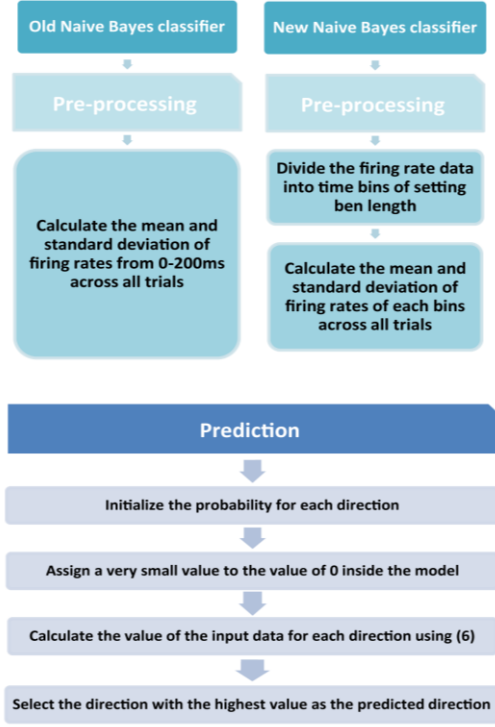


Figure 2. Classification algorithm

In (2) we can find that the core of the comparison is $P(a|Y)$. As mentioned in [5], the Gaussian distribution would usually act as a probabilistic model for the neuron spike. Therefore, $P(a|Y)$ can then be calculated by the following equation (3) [6].

$$P(a_k|Y_i) = g(a_k, \mu_{Y_i}, \sigma_{Y_i}) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(a_k - \mu_{Y_i})^2}{2\sigma_{Y_i}^2}} \quad (3)$$

In our algorithm, on both sides of the equation were taken as the natural logarithm for computational convenience. By ignored the constant $\log_e \frac{1}{\sqrt{2\pi}}$, we only need to compare (4):

$$\log_e P(X|Y_i)P(Y_i) \equiv \log_e P(Y_i) + \sum_{j=1}^m \left(-\frac{(a_j - \mu)^2}{2\sigma^2} - \log_e \sigma \right) \quad (4)$$

Directly computing the mean and standard deviation of all data to obtain $P(a|Y)$ may destroy the spike's characteristics resulting in poor accuracy rate, due to the different situations of neurons. We improved the first version algorithm by reprocessing the pre-movement neuron activity into time bins and got better results. It allows retaining as much of the time-related data features as possible.

Fig. 2 shows the algorithms used for classification and the differences between the old and new versions. The main difference between these two algorithms is in the pre-processing part. New NB will get an $8 \times 98 \times \text{bin}$ number matrix after pre-processing, which means 8 directions, 98 neurons, and the number of time bins. But old NB only gets an 8×98 matrix. Old NB takes a rough data processing interval of 0-200ms because the greyscale display of the average firing rate matrix showed a clear boundary at around 200ms. We extend the period to 0-300ms in new NB and get better accuracy.

B. Position Prediction

We utilized a 2-layer Feedforward Neural Network (FNN) to model the relationship between spike trains during movement and real-time hand position. FNN is an artificial neural network where every neuron in one layer is connected to every neuron in the next layer via weighted connections. The network optimizes these weights using gradient descent to fit a linear polynomial mapping between input and output[7]. The network structure is shown in Fig. 1.

The tanh activation function is used, with mean squared error (MSE) as the loss function. 8 regressors are trained for each direction based on neural activity. The root mean squared error (RMSE) is used to evaluate prediction performance.

Hyperparameters are tuned using grid search within a specified range. The optimal values are found to be a learning rate of 0.001 and 500 training epochs. The layer number and sizes are illustrated in Figure 1, which reflects the network complexity, a factor affecting the results.

III. EXPERIMENTS AND RESULTS

A. Dataset and Preprocessing

Only the provided monkey hand movement dataset is used for training and testing our method.

The raw spike trains are divided into multiple bins with pre-defined length. Subsequently, the firing rate of each bin is calculated by dividing the number of spikes by the bin length. L2 normalization is performed to the series of firing rates before entering the classifier and regressor.

B. Comparative Analysis with Alternative methods

In this report, the dataset is split into a 1:1 ratio of training and test sets. Performance is evaluated based on RMSE, run time, and predicted trajectory shape. The ideal method should have low RMSE and short run time while ensuring similar trajectory shape to the real trajectory. All the experiments are conducted on MATLAB 2019a on a computer.

In order to assess the influence of different classifiers and regressors on the results, we introduced multiple models to perform the task, including, NB, SVM, LSTM[8], and FNN. Specifically, NB and SVM are tested for performing direction classification, while SVR, FNN, and LSTM are tested for regression. All the models are evaluated with their optimal hyperparameters. The comparing models are built using MATLAB toolbox. The results are shown in Table 2 and Fig. 3. Overall, LSTM is the best model that achieves the lowest RMSE. At the same time, it provides an excellent fit to the arc of the real trajectory. However, it requires an excessively long running time, which mainly results from its training. For classification, SVM achieves lower RMSE with a higher cost of time, suggesting that SVM classifies the direction better. The difference in performance among models is relatively small, except for LSTM's running time. In addition, although the combination of Naive Bayes and FNN we chose has the highest RMSE, it can complete the training and prediction process in only 23 seconds, possibly due to the simple structures of Naive Bayes and 2-layer FNN.

TABLE I. RESULTS OF COMPARATIVE ANALYSIS

Comparative Analysis				
Classifier	Regressor	Bin Length [classifier, regressor]	RMSE	Run Time
Naive Bayes	SVR	[100, 90]	26.35916	79s
Naive Bayes	FNN	[100, 90]	26.02985	23s
Naive Bayes	LSTM	[100, 20]	24.85559	1854s
SVM	SVR	[50, 90]	25.30947	138s
SVM	FNN	[50, 90]	25.02332	96s
SVM	LSTM	[50, 20]	23.4772	1836s

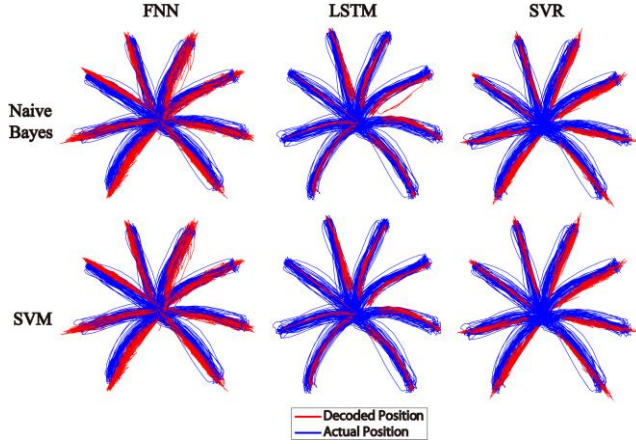


Figure 3. Predicted trajectory of different methods. Row refers to different regressors, while column refers to different classifiers.

C. Exploring the Factors Affecting Direction Classification

For testing the influence of the input parameters and the improvement of the new NB model, the accuracy rate under different bin length in two versions of NB classifier were recorded and further compared to the performance of SVM classifier.

The period of 0 – 200ms was first chosen as the input of direction classifier. Subsequently, it was observed that increasing the length of the pre-movement period used as input for the classifier resulted in an immediate improvement in accuracy. The variation of the information and features would rise at the latest end time. The new NB has better performance than the old one in the setting time and bin length. Compared to the old NB classification method, the improved algorithm retains more features of the data and the accuracy of the prediction increases as a result.

In the old classifier, there were always a few close directions that could be easily misjudged. As shown in Fig.4 (a), the accuracy rate for direction 3 is very high but the accuracy rate for directions 2 and 4 is extremely low. This is probably because the old classifier blurs the significant differences between the classes when extracting features. The new algorithm significantly improves prediction accuracy in some directions such as 2 and 4. It proved that the time bin can retain more important features to increase the classification accuracy rate.

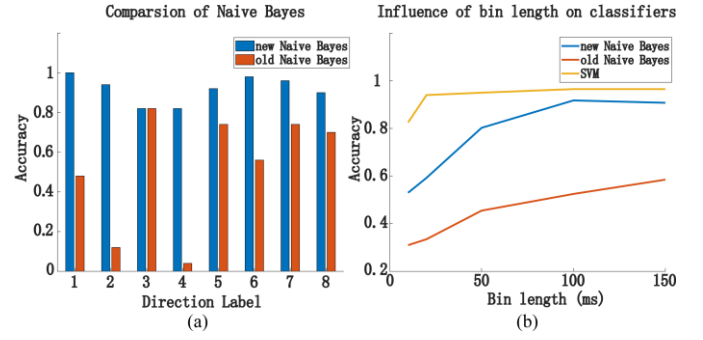


Figure 4. (a) Comparison of the accuracy rate of various directions of the Naïve Bayes before and after improved. Period: 0-300ms, bin length: 100ms. (b) Comparison of the performance between different tested classifiers, with the influence of bin lengths. Period: 0-300ms.

SVM was introduced for the classification performance comparison with the NB solution. In Fig.4 (b), across all choices of bin length, SVM showed a better accuracy rate than both the old and new NB classifier. Additionally, there was smaller influence of bin length on SVM whereas new NB had a substantial increase in accuracy for bin length from 10ms to 100ms. The general low accuracy at short bin length was possibly caused by the resolution of the original temporal information across trials was not accurate enough to support 10ms bin division.

D. Exploring the Factors Affecting Position Regression

There are other factors that may affect position regression, including bin length and model complexity. Among them, the bin length mainly affects the calculation of the firing rate. Short bin lengths result in unsmooth and sparse firing rates, while long bin lengths lead to blurred time domain information. Model complexity, determined by the number of layers and nodes, affects the model's fitting ability and computational requirements. Generally, the more complex the model, the stronger the fitting ability, and the greater amount of calculation, but an overly complex model may also lead to overfitting[7]. Therefore, we investigated the impact of bin length from 10ms to 100ms. We also compared the performance of 2-layer, 3-layer, and 4-layer FNN with hidden layer sizes ranging from 16 to 256. For more than 2 layers FNN, the size of the first hidden layer equals the hidden layer size, while the size of the following hidden layer is half of the previous layer. All the experiments were conducted under the true moving direction, so as to control variables.

Fig. 5 shows the results of the explorations. Here, to speed up the process, we set the batch size of the LSTM input to 10 instead of the optimal value 1. As shown in Fig. 5 (a), except LSTM, all models show an overall downward trend with the increase of bin length and reach the lowest point at a bin length of 90ms. This can be explained by the bin length of 90ms maintaining a good balance between the firing rate smoothness and temporal information. Noting that though SVR achieves the lowest RMSE at most points, the 2-layer FNN delivers a slightly lower RMSE at the point of 90. This indicates that the 2-layer FNN is a better choice to achieve both low RMSE and short running time.

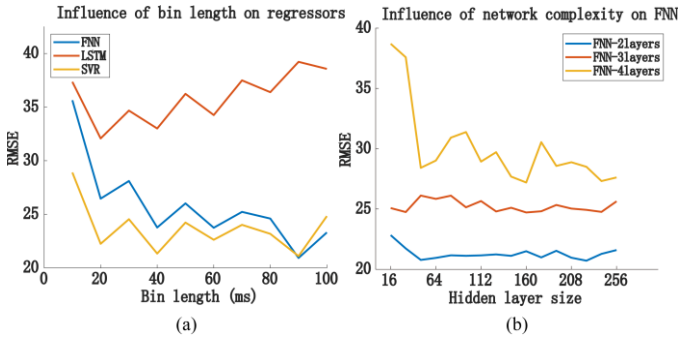


Figure 5. (a) Influence of bin length when using different regressors. (b) Influence of network complexity, x-axis explores the hidden layer size, while different lines explore the number of layers.

In terms of the network complexity, as shown in Fig. 5 (b), the increasing number of hidden layers leads to a higher RMSE, indicating possible overfitting. Besides, the performance of FNN fluctuates with changes in hidden layer size. The results suggest that a 2-layer FNN with a hidden layer size of 48 should be chosen to solve the problem.

IV. DISCUSSION

In this report, we solve the problem of predicting monkey hand positions based on recorded real-time neural activity by dividing the problem into separated classification and regression tasks. Naive Bayes Classifier and Feedforward Neural Networks regressor were introduced for corresponding tasks, which allow achieving low root mean squared error in a short running time. Predicted trajectories are drawn and observed to closely resemble true trajectories.

For optimizing the decoding performance, program structures are refined and various model combinations are evaluated for both classification and regression tasks. The experiments were also conducted for feature selections in direction classifier and hyperparameter tuning in regressor. The most significant improvement from the exploration comes from the decision of the input bin length of the firing rate and the complexity of the models. Also, the direction prediction accuracy was enhanced by retaining more features in the Naïve Bayes classifier. This led to a significant reduction in position error when combined with the corresponding regressors. During the preprocessing stage, the removal of high firing neurons from the spikes input was tested, and the modified dataset was evaluated in the direction classifier. However, this had minimal impact on the outcome, possibly due to a compensatory effect resulting from the reduction in input features and the potential negative impact of highly active neurons. Additionally, the lack of information regarding the specific brain region for recording and other environmental factors and behavior stability may influence the preprocessing outcome[3]. These results suggest that moderate bin length and model complexity as well as simple features are sufficient for solving the problem, indicating the strong mapping relationship between the hand movement and neural activity. It is also worth noting that the value of certain parameter in the models, such as the number of features for

posterior probability calculation in classifier and the hidden layer size in the regressor are determinate for both the running time and the prediction performance. When making decisions about the final solution to the decoding problem, there is a trade-off between the accuracy and the running time. However, most experiments showed that when parameters fell within an appropriate range, there was only a small variation in the root mean square error (RMSE), while a slight further adjustment to some parameters resulted in a negligible improvement in prediction accuracy but a significant increase in running time.

Moreover, beyond the mentioned improvements, extracting and combining meaningful features through some mathematical methods may contribute to find more stable and effective features to improve the decoding performance. For example, extracting the meaningful neural variability across trials[9] and the stable neural manifolds of the neural population dynamics[3]. These provide directions for further enhancing decoding performance in the future.

ATTRIBUTIONS

Yilan Luo and Chuqi Wang developed the direction Classifier, Tao Liu and Hualong Zhang developed the Regressor. Tao Liu, Yilan Luo and Chuqi Wang conducted experiments and wrote the paper.

REFERENCES

- [1] N. D. Skomrock et al., "A Characterization of Brain-Computer Interface Performance Trade-Offs Using Support Vector Machines and Deep Neural Networks to Decode Movement Intent," (in English), *Front. Neurosci.*, Original Research vol. 12, 2018-October-24 2018, doi: 10.3389/fnins.2018.00763.
- [2] J. I. Glaser, A. S. Benjamin, R. H. Chowdhury, M. G. Perich, L. E. Miller, and K. P. Kording, "Machine Learning for Neural Decoding," *eneuro*, vol. 7, no. 4, pp. ENEURO.0506-19.2020, 2020, doi: 10.1523/eneuro.0506-19.2020.
- [3] J. A. Gallego, M. G. Perich, R. H. Chowdhury, S. A. Solla, and L. E. Miller, "Long-term stability of cortical population dynamics underlying consistent behavior," *Nat. Neurosci.*, vol. 23, no. 2, pp. 260-270, 2020/02/01 2020, doi: 10.1038/s41593-019-0555-4.
- [4] A. H. Jahromi and M. Taheri, "A non-parametric mixture of Gaussian naive Bayes classifiers based on local independent features," in *2017 Artificial Intelligence and Signal Processing Conference (AISP)*, 25-27 Oct. 2017 2017, pp. 209-212, doi: 10.1109/AISP.2017.8324083.
- [5] B. M. Yu, G. Santhanam, M. Sahani, and K. V. Shenoy, "Chapter 7 - Neural Decoding for Motor and Communication Prostheses," in *Statistical Signal Processing for Neuroscience and Neurotechnology*, K. G. Oweiss Ed. Oxford: Academic Press, 2010, pp. 219-263.
- [6] H. Kamel, D. Abdulah, and J. M. Al-Tuwaijari, "Cancer Classification Using Gaussian Naive Bayes Algorithm," in *2019 International Engineering Conference (IEC)*, 23-25 June 2019 2019, pp. 165-170, doi: 10.1109/IEC47844.2019.8950650.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [9] M. M. Churchland, B. M. Yu, S. I. Ryu, G. Santhanam, and K. V. Shenoy, "Neural Variability in Premotor Cortex Provides a Signature of Motor Preparation," *The Journal of Neuroscience*, vol. 26, no. 14, pp. 3697-3712, 2006, doi: 10.1523/jneurosci.3762-05.2006.